

Experiment 15

Question: Experiment to understand the data preprocessing in Data science.

Aim

To understand the steps of data preprocessing in data science.

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

def exp_15():
    from sklearn.preprocessing import StandardScaler, LabelEncoder
    from sklearn.model_selection import train_test_split
    rng = np.random.default_rng(8)
    n = 120
    df = pd.DataFrame({
        "Age": rng.integers(21, 60, n).astype(float),
        "Salary": rng.normal(55000, 12000, n).round(0),
        "Dept": rng.choice(["IT", "HR", "Sales"], n),
        "Gender": rng.choice(["Male", "Female"], n),
        "Bought": rng.choice([0, 1], n),
    })
    df.loc[rng.choice(n, 8, replace=False), "Age"] = np.nan           # missing
    df.loc[rng.choice(n, 3, replace=False), "Salary"] = 400000       # outliers
    df = pd.concat([df, df.iloc[:4]], ignore_index=True)             # duplicates
    print("1. Raw data shape:", df.shape)
    print("   Missing:", df.isnull().sum().to_dict(), "| Duplicates:", df.duplicated().sum())

    df = df.drop_duplicates()                                       # 2. duplicates
    print("2. After removing duplicates:", df.shape)

    df["Age"] = df["Age"].fillna(df["Age"].median())               # 3. missing values
    print("3. Missing after imputation:", int(df.isnull().sum().sum()))

    q1, q3 = df["Salary"].quantile([0.25, 0.75])                  # 4. outliers (IQR cap)
    iqr = q3 - q1
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    print(f"4. Salary outliers capped to [{low:.0f}, {high:.0f}]:",
          int((df['Salary'] < low) | (df['Salary'] > high)).sum()), "values")
    df["Salary"] = df["Salary"].clip(low, high)

    df["Gender"] = LabelEncoder().fit_transform(df["Gender"])      # 5. encoding
    df = pd.get_dummies(df, columns=["Dept"], drop_first=True, dtype=int)
    print("5. Columns after encoding:", list(df.columns))

    X, y = df.drop(columns="Bought"), df["Bought"]                # 6. split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y)
    print("6. Train shape:", X_train.shape, "| Test shape:", X_test.shape)

    scaler = StandardScaler()                                     # 7. scaling
    cols = ["Age", "Salary"]
    X_train[cols] = scaler.fit_transform(X_train[cols])           # fit on train only
    X_test[cols] = scaler.transform(X_test[cols])
    print("7. Scaled train mean (Age, Salary):", X_train[cols].mean().round(3).tolist())
    print("   Scaled train std (Age, Salary):", X_train[cols].std(ddof=0).round(3).tolist())
    print("\nFirst 5 rows of preprocessed training data:\n", X_train.head().round(3))

exp_15()
```

Output

1. Raw data shape: (124, 5)
Missing: {'Age': 8, 'Salary': 0, 'Dept': 0, 'Gender': 0, 'Bought': 0} | Duplicates: 4
2. After removing duplicates: (120, 5)
3. Missing after imputation: 0
4. Salary outliers capped to [25820, 86155]: 4 values
5. Columns after encoding: ['Age', 'Salary', 'Gender', 'Bought', 'Dept_IT', 'Dept_Sales']
6. Train shape: (96, 5) | Test shape: (24, 5)

```
7. Scaled train mean (Age, Salary): [-0.0, 0.0]
   Scaled train std (Age, Salary): [1.0, 1.0]
```

First 5 rows of preprocessed training data:

	Age	Salary	Gender	Dept_IT	Dept_Sales
58	-0.305	1.538	1	0	0
19	-0.020	-1.890	1	0	1
4	-1.161	1.595	0	0	1
15	-0.400	0.474	0	0	1
7	1.121	-0.393	0	0	1